Vision — звіт статичного аналізу безпеки (SAST)

Продукт: Vision — візуальні перевірки для QA мобільних застосунків
Коміт: `4ac0ca9` · **Дата:** 2026-11-09
Скоуп: поточне дерево репозиторію: `backend/`, `backend/prisma/`, `frontend/` (зокрема CLI/MCP), `sdk-java/`, `scripts/`, Dockerfile-и, compose-файли, конфіги nginx
Мета: готовність до SAST-сканування перед перенесенням у банківський контур

1. Висновок

Код готовий до SAST-сканування банку. Статичний аналіз не знаходить жодної підтвердженої вразливості, а вразливих залежностей немає.

Перевірка	Інструмент	Результат
Вразливі залежності, prod і dev, 3 npm-пакети	`npm audit`	**0** у кожному з 6 прогонів
SAST: TypeScript, JavaScript, React, Java, Python, Dockerfile, bash, YAML	semgrep, 1081 правило, 980 файлів	**0** підтверджених знахідок (1311 спрацювань розібрано вручну, розділ 3)
SAST: Python — CLI і службові скрипти	bandit	**0** High, **0** підтверджених Medium
Shell-скрипти	shellcheck	**0** проблем безпеки
Секрети в дереві	detect-secrets, правила секретів semgrep	**0** справжніх секретів
Ін'єкції SQL, XSS, `eval`, командні ін'єкції	ручна перевірка	не знайдено

2. Захисні механізми, підтверджені аналізом

- **Ін'єкції SQL.** Prisma викликається лише через ORM і параметризовані шаблони `$queryRaw` / `$executeRaw`, а `$queryRawUnsafe` і `$executeRawUnsafe` не використовуються.
- **XSS.** У коді інтерфейсу немає `dangerouslySetInnerHTML`, `innerHTML` чи `eval`. CSP в nginx і API: `default-src 'self'; script-src 'self'; frame-ancestors 'none'; object-src 'none'`. HSTS видається на запити по https.
- **Командні ін'єкції.** У коді продукту немає `child_process`, а CLI і скрипти передають команди списком аргументів, без shell.
- **Path traversal.** Знімок за іменем видається лише з теки знімків (`hdd.service.ts`, `getImagePath`), а маршрут перевіряється тестом.
- **ReDoS.** Регулярні вирази з правил користувачів виконує RE2 з лінійним часом (`linear-regex.ts`).
- **LDAP-ін'єкції.** Email у фільтрі екранується, і символ `$` не розбирається як шаблон заміни.
- **Криптографія.** Секрети в базі шифруються AES-256-GCM, а при розшифруванні приймається лише повний 16-байтовий тег автентифікації. API-ключі зберігаються як хеш.
- **Секрети запуску.** API не стартує зі слабкими, однаковими чи прикладовими `JWT_SECRET` і `VISION_SECRET_KEY`.

- **Образи.** Базові образи закріплено тегом і digest `sha256`, а процеси працюють від непривілейованого користувача (`USER node`, `USER nginx`).
- **Масове присвоєння.** Глобальний ріре пропускає до обробника лише поля, оголошені в DTO.

Зміни в коді за підсумками аналізу

Що виправлено	Файл	Тест
Розшифрування AES-GCM приймало укорочений тег автентифікації (semgrep `gcm-no-tag-length`)	`backend/src/shared/secret-box.ts`	`secret-box.spec.ts` — значення з тегом, обрізаним до 4 байт, не розшифровується
Символи `$&`, `$\``, `$'` в email змінювали фільтр LDAP в обхід екранування	`backend/src/users/ldap/ldapusers.service.ts`	`ldapusers.service.spec.ts` — фільтр містить рівно введений email

Нові тести падали на коді до виправлення. Уже збережені секрети після зміни читаються як і раніше.

3. Розбір спрацювань semgrep

Кожне спрацювання переглянуто вручну. Чотири з рівнем ERROR — хибні:

- `useless-ternary` у `backend/src/backups/backups.service.ts:97`: тернарний вираз `secret ? 0o600 : 0o644` дає різні права файла, тож він не зайвий;
- `dangerous-subprocess-use-audit` у `frontend/public/cli/vision.py:123`, `scripts/seed-insights-demo.py:295` і `scripts/verify.py:122`: команди задано в коді й передаються списком, без shell.

Група правил	Кількість	Висновок
Стиль і i18n: `i18next-key-format`, `react-props-spreading`, `missing-template-string-indicator`, `jsx-not-internationalized`, `no-stringify-keys`, `string-concat-in-list`, `useless-ternary`, `useless-cat`, `mui-snackbar-message`	1109	не стосується безпеки; формат ключів словника свідомий (ADR-0017)
`package-dependencies-check` — діапазони версій у `package.json`	86	хибне: lock-файли з `integrity`, збірка через `npm ci`
bash: змінні без лапок (рівень INFO)	39	не експлуатується: внутрішні шляхи й порти; частина — сторонні `gradlew` і `wait-for-it.sh`
`path-join-resolve-traversal`, `detect-non-literal-fs-filename`	32	хибне: 26 — тести й тестовий стенд; `hdd.service.ts` перевіряє вихід за теку; `backups.service.ts` бере лише сталі імена файлів
`generic-api-key`	15	хибне: тестові дані й приклади в документації; приклад ключа в `sdk-java/README.md` нечинний, бо API-ключі зберігаються лише хешем, а відкриті ключі анулюються при старті

Група правил	Кількість	Висновок
`detect-non-literal-regexp`	6	хибне: на сервері вираз лише перевіряється на синтаксис, а виконує його RE2; у браузері — підрахунок збігів у вікні автора виразу
`dynamic-urllib-use-detected`, `dangerous-subprocess-use-audit`	8	хибне: адреса API — з локальної конфігурації користувача CLI; команди — списком без shell
`missing-image-version`	4	хибне: образ задано через `ARG` з тегом і digest
`html-in-template-string`, `unsafe-dynamic-method`	6	тести й скрипт збірки іконки
`no-replaceall` (правило сумісності зі старими браузерами), `avoid-platform-with-from`, `remove-package-cache`	4	не стосується безпеки: конфіг збірки виконується на Node 24, збірка на `BUILDPLATFORM` свідома, apt-кеш видаляється
`github-actions-mutable-action-tag`	2	поза скоупом: CI не входить у поставку
Разом	**1311**	

4. Bandit, shellcheck, секрети

- **bandit** (3305 рядків Python): 73 спрацювання, з них High — 0, і 53 припадають на тести (`scripts/tests/`).
- `B608` (SQL з рядків) у `seed-insights-demo.py` — хибне: кожен ідентифікатор перевіряється як UUID до вставки, а скрипт призначений лише для демо-даних.
- `B310` (схема URL) у CLI — адреса з власної конфігурації користувача.
- **shellcheck** (рівень warning і вище): 7 зауважень, жодне не стосується безпеки (`SC2148`, `SC2155`, `SC2010`).
- **detect-secrets**: 123 спрацювання, і всі — назви змінних, заглушки в `.env.example`, облікові дані локальних моків LDAP/OIDC, тестові дані або тексти інтерфейсу. Справжніх секретів немає.

5. Методологія та інструменти

Інструмент	Версія і налаштування
semgrep	1.184.0, офлайн, без метрик; офіційний репозиторій правил `semgrep-rules` (коміт `16b9e6d`): javascript, typescript, java, python, dockerfile, bash, yaml, html, json, generic/secrets — надмножина `p/owasp-top-ten`, `p/nodejs`, `p/typescript`, `p/react`, `p/secrets`
bandit	1.9.4 — `frontend/public/cli`, `scripts`
shellcheck	0.11.0, рівень warning — `scripts/*.sh`, `scripts/lib/*.sh`
detect-secrets	усі відстежувані файли репозиторію
npm audit	`backend`, `backend/prisma`, `frontend`; окремо prod і всі залежності

Сирі виводи: `audit/raw/*-2026-11-09.json`.

Обмеження. Середовище аналізу не дозволило завантажити hadolint, trivy, gitleaks і osv-scanner. Їх замінили правила semgrep для Dockerfile і секретів, detect-secrets і ручна перевірка Dockerfile-ів. Історію git, транзитивні залежності Gradle і зібрані Docker-образи не перевіряли.

Інструменти банку (SonarQube, Checkmarx, Fortify тощо) мають власні правила, але перевіряють ті самі класи проблем.